



Architecture Hexagonale – Backend Spring Boot

1. Objectif du document

Ce document décrit en détail l'**architecture hexagonale (Ports & Adapters)** adoptée pour la partie backend de l'application de gestion des flux financiers.

L'objectif est de : - Garantir une **forte séparation des responsabilités** - Maximiser la **testabilité** - Rendre le système **évolutif et maintenable** - Découpler totalement le cœur métier des frameworks et des technologies

Ce choix architectural est volontairement aligné avec un **niveau architecte senior / entreprise**.

2. Pourquoi l'architecture hexagonale ?

Dans une application financière, le **métier est la valeur principale**. Il doit : - Survivre aux changements technologiques - Être indépendant du web, de la base de données, de la sécurité - Pouvoir être testé sans infrastructure

L'architecture hexagonale permet : - D'inverser les dépendances - De protéger le domaine métier - D'éviter l'effet "framework-driven design"

Le framework (Spring) est un détail, le métier est roi.

3. Principe fondamental

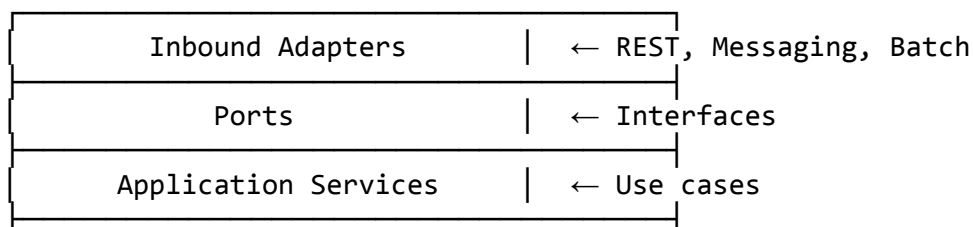
Le cœur de l'application (**le domaine**) ne dépend de **rien d'externe**.

Les dépendances vont toujours :

Extérieur → Intérieur

Jamais l'inverse.

4. Vue globale des couches



Domain	← Métier pur
Ports	← Interfaces
Outbound Adapters	← DB, APIs, Mail

5. Le Domaine (Core Métier)

5.1 Rôle

Le domaine représente la **logique métier pure**.

Il ne connaît : - Ni Spring - Ni JPA - Ni REST - Ni JWT

5.2 Contenu

- **Entities** : Transaction, Account, Budget, Category
- **Value Objects** : Money, Period, DateRange
- **Domain Services** : règles métier complexes

5.3 Règles

- Aucune annotation technique
 - Aucune dépendance vers l'extérieur
 - Code 100% testable avec JUnit
-

6. Couche Application (Use Cases)

6.1 Rôle

La couche application orchestre les **cas d'usage**.

Exemples : - Enregistrer une transaction - Calculer le solde mensuel - Générer un rapport

6.2 Responsabilités

- Coordination du domaine
- Gestion des transactions
- Contrôle fonctionnel

6.3 Ce qu'elle ne fait PAS

- Pas de règles métier complexes
- Pas d'accès direct à la base
- Pas de logique HTTP

7. Ports (Interfaces)

Les **ports** sont des interfaces qui définissent les contrats.

7.1 Ports Inbound

- Exposent les cas d'usage
- Utilisés par les adaptateurs entrants

Exemple : - CreateTransactionUseCase - GetBalanceUseCase

7.2 Ports Outbound

- Définissent les besoins du domaine vers l'extérieur

Exemples : - TransactionRepositoryPort - NotificationPort - UserIdentityPort

8. Adapters Inbound

8.1 Rôle

Les adaptateurs entrants traduisent une interaction externe vers un cas d'usage.

8.2 Exemples

- REST Controllers (Spring MVC)
- Event Consumers
- Batch Jobs

8.3 Responsabilités

- Mapping DTO → Domain
 - Validation technique
 - Gestion HTTP
-

9. Adapters Outbound

9.1 Rôle

Ils implémentent les ports sortants.

9.2 Exemples

- JPA Repositories
- Clients REST
- Clients SMTP

9.3 Règle clé

Un adaptateur peut être remplacé sans toucher au domaine.

10. Infrastructure & Frameworks

Spring Boot est utilisé comme : - Conteneur d'injection - Moteur transactionnel - Support technique

Mais **n'impose jamais** sa logique au métier.

11. Gestion des dépendances

- Le domaine ne dépend de rien
- L'application dépend du domaine
- Les adaptateurs dépendent des ports

Cette règle est **non négociable**.

12. Tests & Qualité

- Tests unitaires sur le domaine
 - Tests des use cases sans Spring
 - Tests d'intégration pour les adaptateurs
-

13. Bénéfices clés

- Forte maintenabilité
 - Facilité de test
 - Adaptabilité technologique
 - Lisibilité architecturale
-

14. Conclusion

Cette architecture positionne le backend comme un **système robuste, pérenne et orienté métier**, prêt pour un contexte **fintech / entreprise**.

C'est un choix d'architecture assumé, exigeant, mais extrêmement payant à long terme.

Auteur : Yassin MELLOUKI

Rôle : Backend Architect

Technologie : Spring Boot – Architecture Hexagonale

Date : 2026-02-07